

bornal

kit for bitcoin related integration test tools

qlrd - Krux contributor

Who am I?

Who am I?

- Har Har Mahadev(s), I'm qlrd (it means “qualquer dev” in Brazilian Portuguese, which means “any dev”)
- Krux contributor
- today I review more than I code
- When I find the time, I code boral (see Appendix I for what it means).

Source code



Figure 1: <https://github.com/qlrd/bornal-presentation>

Idea summary

- 1. Fast overview on Bitcoin Test Framework
- 2. Middle philosophy on “laboratory problem”
- 3. What some people did and do
- 4. Apendix for curiosity

Overview: Bitcoin Test Framework [1]

Definition 1.

The functional tests [that] test the RPCs. [2]

Overview: Consequence [3]

(...) it takes pretty long. In general they take longer than unit tests. (...) pay some attention to how you write the tests and how many you write.

The shape [4]

```
class MyTest(BitcoinTestFramework):  
  
    def set_test_params(self):  
        self.num_nodes = 2  
  
    def run_test(self):  
        self.connect_nodes(0, 1)  
        self.generate(self.nodes[0], 1)  
        self.sync_all()
```

BitcoinTestFramework is a metaclass and enforce those methods.

A test framework is good for whom?

A definition by its object, POV, beneficiary

- “The functional tests [that] test the RPCs.” [2]: It names **what** is tested
- “(...) from a user’s perspective.” [3]: It names **who is looking**
- Neither name a beneficiary: is good for **whom**?

“Good for whom” implies “at whose cost”.

- slow suites [3];
- minutes of compilation;
- one more repository;
- drift, when the tool moves and the examples lag.

Candidates

- The author of the change already knows what they did.
- the reviewer: made by someone who does **not** trust the description of the change
- other implementations
- the end use: Good **for** them, never good **to** them.

At whose cost?

A framework distributes confidence **and** costs. The two lists of names rarely match.

Everyone builds their own

Bitcoin Core [4]

- every node in `self.nodes` is a bitcoind;
- one build: the tree you are standing in;
- one dialect: authproxy;
- every failure is Core's.

Good for Core.

Liana [5]

C-lightning pyln-testing [6] → revaultd [7] → liana.

Not a Core derivation. A second family.

Liana: a pytest 2nd layer

```
@pytest.fixture
def bitcoind(directory):
    bitcoind = Bitcoind(bitcoin_dir=directory / "bitcoind")
    bitcoind.startup()
    bitcoind.rpc.generatetoaddress(101, ...)
    yield bitcoind
    bitcoind.cleanup()
```

```
@pytest.fixture
def lianad(bitcoin_backend, directory): ...
```

Liana: own differences

- binaries from BITCOIND_PATH, ELECTR5_PATH, LIANAD_PATH: you build them;
- bitcoin_backend is Bitcoin or Electrs, chosen by an env var;
- any RPC is a method: __getattr__ → AuthServiceProxy;
- a fresh directory and a fresh chain per test.

Floresta [8]

Lineage: `Core.p2p.py`, `messages.py`, `key.py` are vendored from `test_framework`.

`prepare.sh` builds `bitcoind`, `utreexod` and `florestad` into a `BINARIES_DIR`.

Floresta: a pytest 2nd layer

```
@pytest.fixture
def node_manager(setup_logging, request):
    manager = FlorestaTestFramework(...)
    yield manager
    manager.stop()

@pytest.fixture
def bitcoind_node(node_manager) -> Node:
    node = node_manager.add_node_default_args(
        variant=NodeType.BITCOIND)
    node_manager.run_node(node)
    return node
```

Similar ways, own differences

	Core	Liana	Floresta
lineage	itself	C-lightning	Core
who runs it	main()	pytest fixtures	pytest fixtures
binaries	the tree	*_PATH env	prepare.sh
RPC	authproxy	__getattr__ proxy	node.rpc
daemons	bitcoind	bitcoind, electrs, lianad	bitcoind, utreexod, florestad
scope	one script	function	function, class

One implementation

- A self-test: the framework asserts what the software does (Core testing Core).
- 2 or more implementations:
 - “me vs the ref”
 - Liana against core and using electrs
 - Floresta against core and using utreexod
 - Borna1: framework to test krux ecosystem (not only krux is the aim) with some possible implementation: bitcoind, bitcoind+electrs (sparrow or other popular client), Floresta/utreexod and Liana

More than two: other issues arise

Ownership

Krux's tree? MicroPython for a K210.

It does not want a bitcoind build in its CI;

Build

daemon	build	language
bitcoind	CMake since v29, Autotools before	C++
electrs	cargo build --locked --release	Rust
florestad, lianad	cargo	Rust
utreeexod, btcd	go build	Go

There is no release binary for every cell of tag $\times$ wallet $\times$ platform $\times$ implementation.

Dialect

	protocol	ping
bitcoind	JSON-RPC 1.0 over HTTP	uptime
electrs [9]	JSON-RPC 2.0 over raw TCP	server.ping
Krux	a PSBT through a QR code	none

One client class no longer fits.

Lifecycle

With one implementation, every node is a peer.

electrs needs a **started** bitcoind. There is an order to start, and the reverse order to stop.

Synchronization

“Synced” means:

- block count, for bitcoind;
- index tip, for electrs;
- lastprocessedblock, for a wallet.

One `sync_all()` becomes `sync_blocks, wait_for_height, wait_wallet_synced`.

The oracle

When electrs and bitcoind disagree, which one is wrong?

When Krux refuses a PSBT that Core accepts, which one is right?

Core, by default. But the test has to say so, not assume it.

Attribution

With one implementation, the bug is yours. With three, whose is it?

AssertionError:

```
'bitcoind' at 127.0.0.1:41733 expected 1 blocks, got 0
```

The message has to name the node, or the failure is nobody's.

The version matrix

One implementation has one version at a time.

Three have a matrix, and the interesting bugs sit in the cells:

```
$> pytest --build-bitcoin 30.2 --build-electrs latest ...
```

Trust

Inside a single-implementation test there is no trust boundary.

With a signer in the picture, the test **is** the boundary.

Trust, made executable

- Core holds the tpub, never the tprv: the coordinator is not trusted with the key;
- blocks are mined to an unspendable address: no wallet has to be trusted for the chain to advance;
- a second node only sees the transaction: the network is a witness.

Definition 2.

A test framework is a trust model made executable.

bornal

Good for whom? [10]

Definition 3.

I did it for me: the tester who sits between implementations and owns none of them.

The wish

I do not want to rebuild my own test framework again and again.

- add it to `pyproject.toml`;
- declare the plugins: `bitcoin-core`, `electrs`, `florestad`, `utreexod`;
- run the install. It will be long the first time;
- then exercise any kind of test without plumbing one by one.

In one sentence [10]

It scaffolds some test structure in a jekyll-like style and lets pytest know how to build bitcoin regtest daemons from source (...) and hands your tests a ready JSON-RPC client, so you write your integration tests, hacking through your preferred bitcoin lib, not plumbing through it.

The workflow

```
$> uv add --dev git+https://github.com/qlrd/bornal.git@<tag>
$> uv run bornal create p2pkh --template wallet-roundtrip

# first time: minutes of cmake and cargo
$> uv run pytest --build-bitcoin 30.2 --build-electrs latest \
    --wallet tests/integration

# next time
$> uv run pytest tests/integration
```

A plugin is three classes [11]

```
[project.entry-points."bornal.daemons"]  
bitcoin-core = "bornal.plugins.bitcoind:CoreCompiler"  
electrs      = "bornal.plugins.electrs:ElectrsCompiler"
```

```
[project.entry-points.pytest11]  
bornal = "bornal.fixtures"
```

Compiler, Daemon, Client. bornal never imports one: it discovers whatever is installed.

Each class answers an issue

- Compiler → **build**;
- Daemon → **lifecycle**;
- Client → **dialect**;
- entry points → **ownership**;
- helpers that name the node → **attribution**;
- the watch-only rule → **trust**.

The Compiler

Cache first, PATH second, source third.

```
wanted = {"revision": revision or "latest", "wallet":  
bool(wallet)}  
if os.path.exists(dest) and not force:  
    if self.read_build_meta(paths) == wanted:  
        return dest
```

bitcoind.build.json remembers how the cache was built.

The Daemon

```
argv = [  
    "-chain=regtest", "-datadir=%s" % self.datadir,  
    "-rpcport=%d" % self.port, "-server=1", ...  
]  
if self._p2p_port is None:  
    argv.append("-listen=0")
```

One datadir per backend. `free_port()` for RPC, p2p and monitoring.

The Client

Two dialects, one interface: `call`, `is_up`, `wait_until_up`.

```
def get_block_count(self) -> int:
    return self.call("getblockcount")

def get_tip(self) -> dict:
    return self.call("blockchain.headers.subscribe")
```

The shape survives

```
class IntegrationTest(ABC):  
  
    def add_backend(self, name, **kwargs): ...  
  
    @abstractmethod  
    def set_test_params(self): ...  
  
    @abstractmethod  
    def run_test(self): ...  
  
    def on_stop_test(self): ...
```

Core's three names.

...pytest drives it

```
@pytest.fixture(scope="module")
def integration_test(request, test_factory):
    test = test_factory(data_dir=...)
    test._on_set_test_params()
    test._on_run_test()
    yield test
    test._on_stop_test()
```

Liana's and Floresta's fixtures. Nodes stay up until the module's last test.

conftest.py

```
class BaseTest(IntegrationTest):
    def set_test_params(self):
        for _ in range(2): # alice, bob
            self.add_backend("bitcoin-core",
                             p2p_port=free_port())

@pytest.fixture(scope="module")
def test_factory(request):
    return BaseTest
```

conftest.py

```
@pytest.fixture
def alice(integration_test):
    return integration_test.backends[0]
```

```
@pytest.fixture
def bob(integration_test):
    return integration_test.backends[1]
```

A test

```
def test_001_connect_p2p(alice, bob):  
    connect_p2p(alice, bob)  
  
def test_003_generate(alice):  
    generate_to_address(alice, UNSPENDABLE_ADDRESS, 1)  
  
def test_004_sync_blocks(alice, bob):  
    sync_blocks(alice, bob)  
    assert_block_count(bob, 1)
```

The file tells a story

The received wisdom: tests should be independent.

A blockchain is a state machine: height only goes up, coinbases mature. So one session per module, state travels between tests.

Run the whole module, not a single test.

bornal.testing

- assert_chain
- assert_block_count
- assert_wallet_info
- assert_wallet_roundtrip
- assert_next_role
- assert_finalized /
not_finalized
- assert_mempool_accepts /
rejects
- assert_send_rawtx_accepts /
rejects
- connect_p2p
- sync_blocks
- wait_wallet_synced
- create_wallet
- get_new_address
- generate_to_address
- COINBASE_MATURITY = 100
- BASE_COINBASE_SUBSIDY = 50

Downstream: krux-integration-tests [12]

Krux is an air-gapped signer: it never touches the network.

- two bitcoin-core nodes: one **coordinates**, the other **witnesses**;
- Core imports the tpub, funds it, builds the unsigned PSBT;
- Krux (the Python module, under pytest) inspects and signs;
- Core finalizes and broadcasts; the witness should see the tx.

A signer's job is mostly to refuse

- wrong purpose, wrong network, wrong policy;
- malformed PSBTs, fabricated previous txs, non-standard sighashes, outputs above inputs.

The happy path is one test. The refusals are the suite.

Keep the Core side watch-only

Passing the tprv to Core would “work”. Do not:

1. **logging**: the RPC client logs every call with its parameters; pytest captures that, CI keeps it;
2. **persistence**: Core writes imported keys to a datadir that is reused;
3. **fidelity**: a test could pass without Krux signing anything.

Where this goes

Not solved yet

- **lifecycle:** electrs cannot be declared from `set_test_params()`;
- **oracle:** Core is still the default answer, by convention;
- **drift:** the tool moves, the examples lag;
- **plugins:** btcd, utreexod, florestad, lianad are TODO.

A hardware-wallet warnet?

Warnet runs scenarios across a **network of nodes**. The analogue for signers:

- FFI bindings to signer firmware: Krux already runs as a Python module under pytest; C cores through ctypes or cffi;
- many signers, one regtest: Core coordinates, electrs indexes, a second node witnesses;
- multisig across vendors, PSBT round-trips, and above all the refusals.

Good for whom, again

For the signer developers who cannot integration-test against real nodes today.

And for whoever has to audit them.

Links

- github.com/qlrd/bornal
- github.com/qlrd/krux-integration-tests
- [the previous talk](#)

Bibliography

Bibliography

- [1] qlrd, “Bitcoin Test Framework (vintenum's study hour).” [Online]. Available: <https://github.com/qlrd/bitcoin-test-framework-presentation>
- [2] A. Chown, “How does bitcoin functional test framework work?” [Online]. Available: <https://bitcoin.stackexchange.com/questions/73932/how-does-bitcoin-functional-test-framework-work>
- [3] F. Jahr, B. Bishop, and M. Folkson, “Bitcoin Core Functional Test Framework.” [Online]. Available: <https://btctranscripts.com/edgedevplus/2019/bitcoin-core-functional-test-framework>
- [4] The Bitcoin Core developers, “test/functional/test_framework.” [Online]. Available: https://github.com/bitcoin/bitcoin/tree/master/test/functional/test_framework
- [5] Wizardsardine, “Liana functional tests (test_framework).” [Online]. Available: https://github.com/wizardsardine/liana/tree/master/tests/test_framework

Bibliography (ii)

- [6] Core Lightning developers, “pyln-testing: Core Lightning's test framework.” [Online]. Available: <https://github.com/ElementsProject/lightning/tree/master/contrib/pyln-testing>
- [7] Revault developers, “revaultd functional tests.” [Online]. Available: <https://github.com/revault/revaultd/tree/master/tests>
- [8] The Floresta developers, “Floresta functional tests.” [Online]. Available: <https://github.com/getfloresta/Floresta/tree/master/tests>
- [9] Electrum developers, “Electrum Protocol Specification.” [Online]. Available: <https://electrum-protocol.readthedocs.io/en/latest/>
- [10] qlrd, “bortal: kit for bitcoin related integration test tools.” [Online]. Available: <https://github.com/qlrd/bortal>
- [11] The pytest developers, “Writing plugins: making your plugin installable by others.” [Online]. Available: https://docs.pytest.org/en/stable/how-to/writing_plugins.html#making-your-plugin-installable-by-others

Bibliography (iii)

- [12] qlrd, “krux-integration-tests.” [Online]. Available: <https://github.com/qlrd/krux-integration-tests>
- [13] R. C. E. dos S. T. Botelho, “A terminologia náutica no Dicionário da Língua Brasileira de Luiz Maria da Silva Pinto,” Dissertação (Mestrado em Linguística Teórica e Descritiva), Faculdade de Letras, Universidade Federal de Minas Gerais, Belo Horizonte, 2011. [Online]. Available: <https://hdl.handle.net/1843/DAJR-8N7H4Q>

Appendix I: embornal [13]

Silva Pinto, *Diccionario da Lingua Brasileira* (Ouro Preto, 1832), entry 76, as transcribed and compared in [13, p. 112]:

Embornal, masc. noun. The bag containing barley or corn into which a beast of burden inserts its muzzle. (Naut. term) Holes in the ship's side to drain water falling onto the deck.

Appendix I: embornal

→Cunha: embornal - bornal masc. noun “cloth bag used to carry provisions, tools, etc.” 1813. Of uncertain origin || Embornal 1813. Embornais masc. noun pl. “(Naval Arch.) opening made in the side of a vessel flush with the deck, for the drainage of wash-down or rainwater” 1813. From Ital. **imbrunali**.

Appendix I: embornal

→Bluteau: Embornàes. (Nautical term) These are holes in the ship's sides, next to the decks, through which water drains from them into the sea. [...] There are other **Embornaes** in the deck's [waterways/scuppers], through which water flows into the hold, from where it is later removed by the pump.

Appendix I: embornal

→Moraes e Silva: EMBORNÁL or Ambornal, masc. noun. Bag in which barley or corn is given to beasts of burden, placed over their muzzles. § Embornáes, naut. term: holes in the ship's side at deck level through which water falling onto the decks drains away; they feature flaps of tarred or oiled cloth through which the water exits. Amaral, 51. See **orndes**.

Appendix I: embornal

→Laudelino Freire: EMBORNAL, masc. noun. From **em** + **bornal**. Bag in which barley or corn is given to beasts of burden, fastened around the animal's mouth; nosebag: "I saw you, a little creature that could fit inside a nosebag, do you hear?" (C. Neto).

Appendix I: embornal

→Aurélio: **embornal** (From Old Catalan **embrunal**, possibly via Spanish **embornal**.) Masculine noun. 1. Naval Arch. An opening made in a vessel's side, flush with the deck, to allow for the drainage of wash-down water or rain.

Appendix I: embornal

→DHPB/CNPq Project database: Both commanders were valiant; and, treating the occasion to which the situation itself led them as a matter of honor, they applied such force to the oars and sails that, on the night following the 9th, they positioned themselves beneath the enemy batteries, showing such disregard for the hail of bullets that, just as the Dutch believed themselves merely under attack, they found themselves boarded; yet, recovering from the initial shock, the Dutch resisted the fury of the blows with such steadfastness that blood was soon running through the scuppers on both sides of the ship.

Bernardo Pereira de Berredo (1749) [1718], *Annaes Historicos do Estado do Maranhão*, Livro V [A00_2517, p. 183].